



AULÃO PRÉ-PROVA 2

AEDS I

30/10/2025

Thiago Venturim

CONTEUDO

01. Dicas e Orientações

05. Matriz e Vetor

02. Breve Recapitulação

06. Alocação Dinamica

03. Bibliotecas para Prova

07. Arquivo

04. Ponteiro

08. Struct



Dicas e Orientações

As provas de AEDs 1 são manuscritas.
Portanto, para treinar, faça no papel, e teste o seu algoritmo mentalmente.

- Ler com atenção as questões.
- Grifar o que está sendo pedido para fazer.
- Comentar seu código.
- Cuidado com tempo.

Pilha (Stack)

- Localização: topo da memória do processo (como mostra o diagrama).
- Uso principal: armazenamento de variáveis locais e chamadas de função.
- Alocação: automática, feita em tempo de compilação quando a função é chamada.
- Organização: LIFO (Last In, First Out — último a entrar, primeiro a sair).

Características:

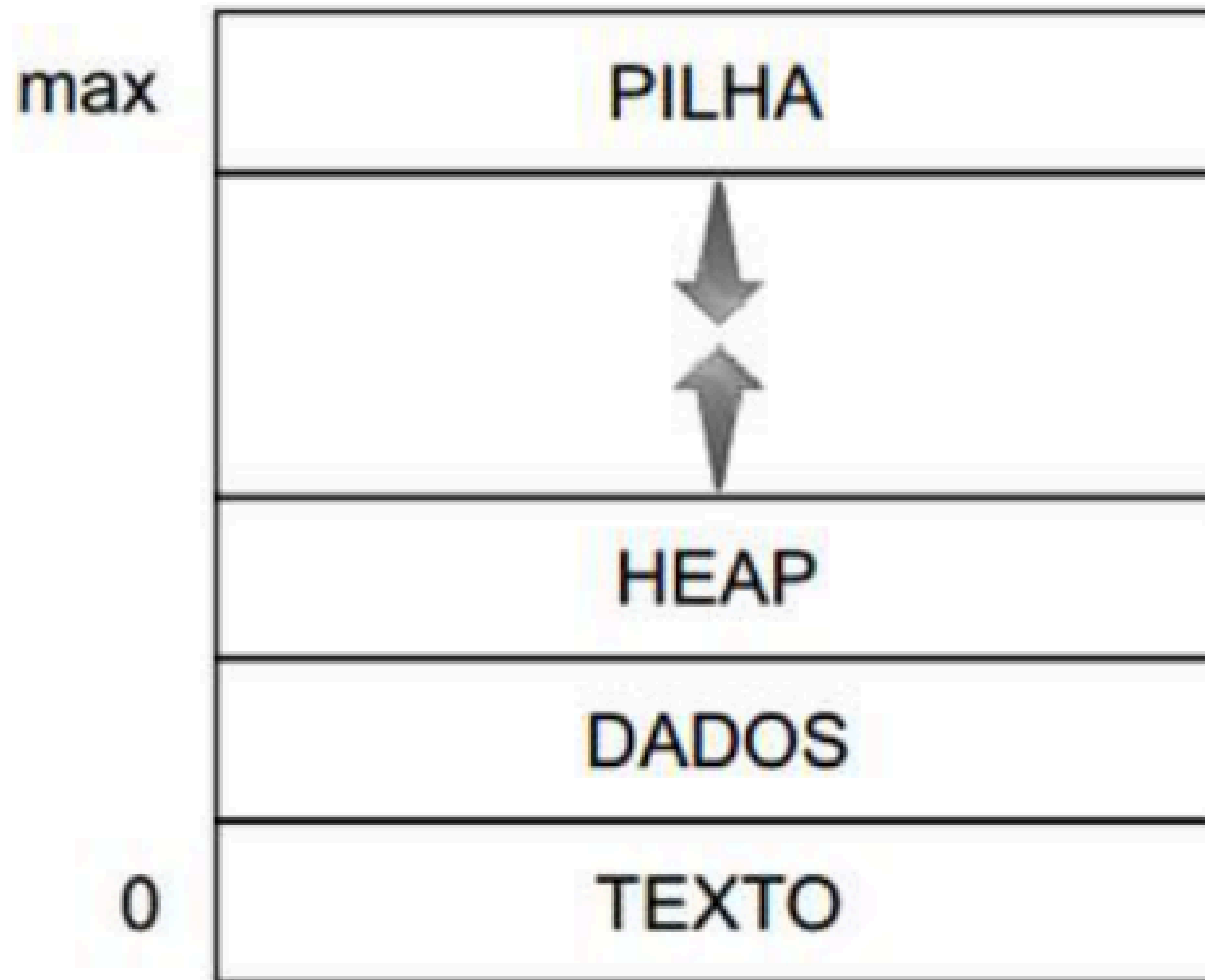
- Muito rápida para alocação e desalocação.
- Limite de tamanho geralmente definido pelo sistema.
- Memória é liberada automaticamente ao sair da função.

Heap

- Localização: região central da memória do processo (abaixo da pilha, como no diagrama).
- Uso principal: armazenamento de memória dinâmica (usando malloc, calloc, realloc em C).
- Alocação: feita em tempo de execução, pelo programador.

Características:

- Mais flexível, mas mais lenta que a pilha.
- Necessita liberação manual usando free() para evitar memory leaks.



Processo em memória

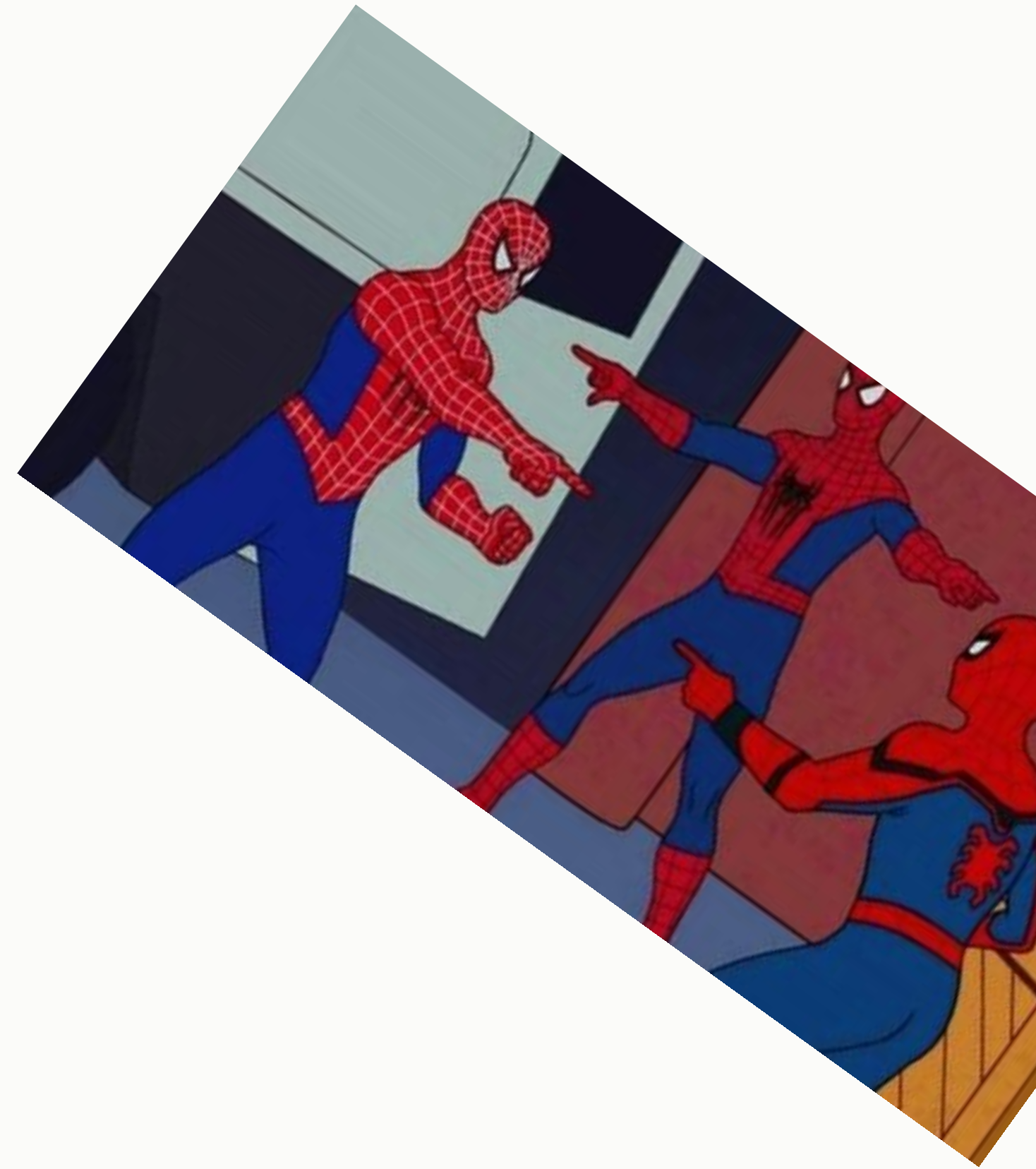
Fonte: Silberschatz (2011, p.57)

Ponteiros em C

- Um ponteiro é uma variável que guarda o endereço de memória de outra variável.
- Permite acessar e manipular dados indiretamente.

Operadores importantes:

- `&` → retorna o endereço de memória da variável
- `*` → acessa o conteúdo armazenado nesse endereço

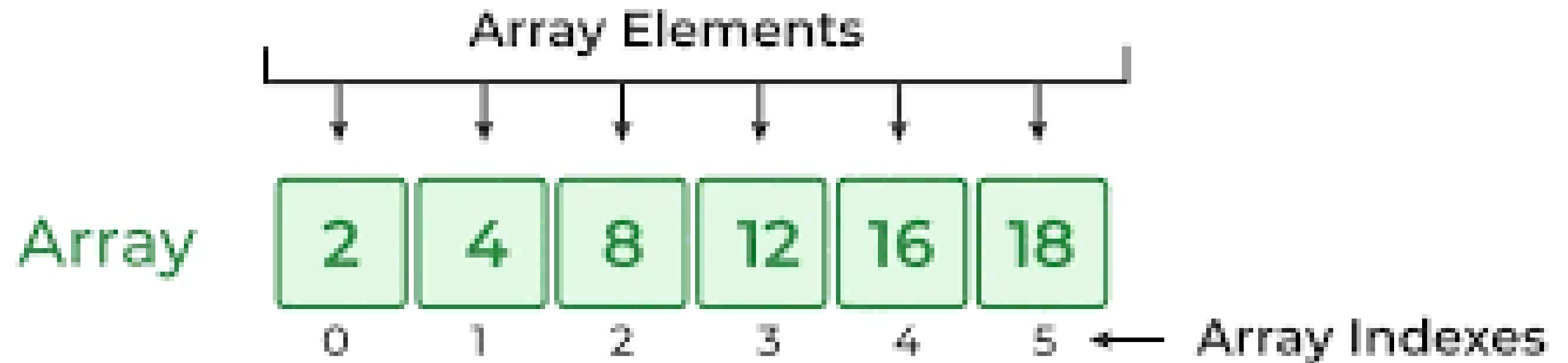


```
1  #include <stdio.h>
2
3  ✓ int main() {
4      int x = 10;
5      int *p;      // Declara um ponteiro para int
6      p = &x;      // Ponteiro p recebe o endereço de x
7
8      printf("Valor inicial de x: %d\n", x);
9
10     (*p)++;      // Incrementa o valor de x usando o ponteiro
11
12     printf("Valor de x apos incremento: %d\n", x);
13
14     return 0;
15 }
16
```

Índices em C

- Em C, arrays e strings são indexados a partir de 0.
- Isso significa que o primeiro elemento está na posição 0, o segundo na posição 1, e assim por diante.

Array in C



Vetor (Array Unidimensional)

- Um vetor é uma coleção de elementos do mesmo tipo, armazenados em posições contíguas da memória.
- Ele é, na prática, um ponteiro que aponta para o primeiro elemento do bloco de memória (alocado na pilha, se declarado de forma estática).

```
● 1  #include <stdio.h>
2  ✓ void lerVetor(int *v, int tamanho) {
3  ✓     for(int i = 0; i < tamanho; i++) {
4      |         printf("Digite o elemento %d: ", i + 1);
5      |         scanf("%d", (v + i));    // aritmética de ponteiro
6      |     }
7  }
8  ✓ void mostrarVetor(int *v, int tamanho) {
9      |     printf("\nElementos do vetor:\n");
10  ✓     for(int i = 0; i < tamanho; i++) {
11      |         printf("%d ", *(v + i)); // acessa valor via ponteiro
12      |     }
13      |     printf("\n");
14  }
15  ✓ int main() {
16      |     int vetor[10];    // vetor estático (na pilha)
17      |
18      |     lerVetor(vetor, 10);    // passa o endereço do primeiro elemento
19      |     mostrarVetor(vetor, 10);
20      |
21      |     return 0;
22  }
```

Vetor (Array Bidimensional)

- Uma matriz é um vetor de vetores (estrutura bidimensional).
- Armazena dados em linhas e colunas.
- Cada elemento é identificado por dois índices: [linha][coluna].
- Assim como o vetor, a matriz é armazenada na pilha (se declarada de forma estática).

```
1  #include <stdio.h>
2
3  void lerMatriz(int *m, int linhas, int colunas) {
4      for(int i = 0; i < linhas; i++) {
5          for(int j = 0; j < colunas; j++) {
6              printf("Elemento [%d][%d]: ", i, j);
7              scanf("%d", (m + i*colunas + j)); // aritmética de ponteiro
8          }
9      }
10 }
11
12 int main() {
13     int mat[2][3];
14     lerMatriz(&mat[0][0], 2, 3); // passa o endereço do primeiro elemento
15     mostrarMatriz(&mat[0][0], 2, 3);
16     return 0;
17 }
18
```

Alocação Dinâmica

Por que usar alocação dinâmica:

- Permite criar variáveis/arrays em tempo de execução.
- Útil quando não se sabe o tamanho da estrutura de dados antes do programa rodar.

Funções principais:

- malloc, calloc, realloc, free.

Diferença entre malloc e calloc

Diferença entre malloc e calloc:

Função Inicialização Sintaxe

malloc = Não inicializa (conteúdo indefinido) = `int *v = (int*)malloc(5 * sizeof(int));`

calloc = inicializa com zero = `int *v = (int*)calloc(5, sizeof(int));`

Cuidado!

Libere o Vetor = `free(v)`

- Memory leaks: memória não liberada.
- NULL: sempre verificar se a alocação funcionou.
- Realocação: usar `realloc` para ajustar o tamanho do bloco.

Arquivo

- Arquivo: armazenamento permanente de dados no disco.
- Fluxo de dados: entrada (leitura) e saída (escrita) via ponteiros de arquivo (FILE *).

Funções Básicas

Função	Uso
fopen	Abrir arquivo
fclose	Fechar arquivo
fprintf	Escrever texto formatado
fscanf	Ler texto formatado
fgets	Ler linha de texto
fputs	Escrever linha de texto

Modos de abertura

Modo	Significado
"r"	leitura
"w"	escrita (apaga conteúdo existente)
"a"	anexar ao final
"rb"	leitura binária
"wb"	escrita binária

Strings <string.h>

- String: sequência de caracteres terminada com \0 (null).
- Em C, strings são arrays de char

Função	Uso
strlen(s)	Retorna tamanho da string
strcpy(dest, src)	Copia string
strncpy(dest, src, n)	Copia até n caracteres
strcat(dest, src)	Concatena strings
strcmp(s1, s2)	Compara strings
strchr(s, c)	Procura caractere
strstr(s, substr)	Procura substring

%s no scanf

- Lê uma palavra (para ao encontrar espaço, tab ou \n).
- Não lê espaços no meio da string.
- Não protege contra overflow, é preciso limitar o tamanho
- O tamanho da string lida não inclui espaços e termina em \0.

`%[^\\n]` no `scanf`

- Lê uma linha inteira até o `\\n` (Enter) ser pressionado.
- Mantém espaços dentro da linha.
- Necessário espaço antes do `%` (" `%[^\\n]`") para ignorar `\\n` residual no buffer.
- Também precisa se preocupar com tamanho do array para evitar overflow:

fgets

- Lê uma linha inteira, incluindo espaços.
- Inclui o \n no final se houver espaço no buffer.
- Protege contra overflow automaticamente (limite definido pelo segundo parâmetro).
- Ideal para leitura de strings completas e seguras.

strcmp

Retorno	Significado
0	Strings iguais
< 0	s1 é menor que s2 (primeiro caractere diferente tem menor valor ASCII)
> 0	s1 é maior que s2

Contador de Vogais

Faça um programa que leia uma string digitada pelo usuário e mostre na tela quantas vogais (a, e, i, o, u, A, E, I, O, U) essa string possui.

Exercício 1: Vetor Invertido com Alocação Dinâmica

Criar um programa que lê um número N , aloca dinamicamente um vetor de N inteiros, preenche esse vetor, inverte a ordem dos seus elementos e, por fim, o exhibe.

Requisitos:

Requisitos:

- O programa deve primeiro perguntar ao usuário o tamanho do vetor (N).
- A memória para o vetor deve ser alocada dinamicamente (usando malloc ou calloc).
- O usuário deve preencher o vetor com N números.
- Você deve criar uma função void inverter_vetor(int *vetor, int tamanho) que recebe o vetor e seu tamanho, e inverte os elementos no próprio vetor (in-place).
- O programa principal (função main) deve imprimir o vetor original e o vetor invertido.
- Não se esqueça de liberar a memória alocada (usando free) antes de terminar o programa.

Dica: Para inverter, você pode usar dois índices (um começando em 0 e outro em N-1) e trocar os elementos até que os índices se encontrem no meio.

Exercício 2: Soma da Diagonal Inferior de uma Matriz

Objetivo: Calcular e exibir a soma de todos os elementos que estão abaixo da diagonal principal de uma matriz quadrada.

Requisitos:

- Defina uma constante para o tamanho da matriz (ex: TAMANHO = 5). A matriz será 5 a 5.
- Crie um procedimento (uma função void) chamado preencher_matriz(int mat[][TAMANHO], int tam) que receba a matriz e seu tamanho, e permita ao usuário preenchê-la com valores inteiros.
- Crie uma função chamada int somar_diagonal_inferior(int mat[][TAMANHO], int tam) que calcula e retorna a soma dos elementos abaixo da diagonal principal.
- A diagonal principal é onde o índice da linha é igual ao índice da coluna ($i == j$). A "diagonal inferior" (ou triângulo inferior) é onde o índice da linha é maior que o índice da coluna ($i > j$).
- Na função main, declare a matriz, chame o procedimento de preenchimento e, em seguida, chame a função de soma, exibindo o resultado final.

Exercício 3: Verificador de Palíndromo

Objetivo: Criar um programa que lê uma string (palavra ou frase) e determina se ela é um palíndromo.

Definição: Um palíndromo é uma string que é lida da mesma forma de trás para frente, desconsiderando espaços e maiúsculas/minúsculas (embora, para uma versão mais simples, você possa considerar espaços e diferenciar maiúsculas/minúsculas).

Requisitos:

- O programa deve ler uma string do usuário.
- Crie uma função `int eh_palindromo(char *str)` que recebe a string.
- A função deve retornar 1 (verdadeiro) se a string for um palíndromo e 0 (falso) caso contrário.
- Na função main, o programa deve imprimir "É um palíndromo" ou "Não é um palíndromo" com base no retorno da função.

Dica: Obtenha o tamanho da string (com `strlen`). Use dois índices, `inicio = 0` e `fim = tamanho - 1`. Compare `str[inicio]` com `str[fim]`. Se forem diferentes, retorne 0. Se forem iguais, incremente `inicio` e decmente `fim`, e continue o loop enquanto `inicio < fim`. Se o loop terminar, é um palíndromo.

Desafio: Matriz Aleatória Única em Arquivo

Objetivo: Gerar uma matriz 4×4 preenchida com números aleatórios únicos (sem repetição) dentro do intervalo de 125 a 305 (inclusive) e salvar essa matriz em um arquivo de texto.

Requisitos:

- A matriz deve ser 4×4 .
- Os números devem ser gerados aleatoriamente no intervalo [125, 305].
- Restrição crucial: Nenhum número pode se repetir dentro da matriz.
- O programa deve criar (ou sobrescrever) um arquivo chamado matriz_aleatoria.txt.
- A matriz deve ser impressa no arquivo de forma formatada (como uma matriz), por exemplo:

Desafio: Matriz Aleatória Única em Arquivo

Objetivo: Gerar uma matriz 4 por 4 preenchida com números aleatórios únicos (sem repetição) dentro do intervalo de 125 a 305 (inclusive) e salvar essa matriz em um arquivo de texto.

Requisitos:

- A matriz deve ser 4 - 4.
- Os números devem ser gerados aleatoriamente no intervalo [125, 305].
- Restrição crucial: Nenhum número pode se repetir dentro da matriz.
- O programa deve criar (ou sobrescrever) um arquivo chamado `matriz_aleatoria.txt`.
- A matriz deve ser impressa no arquivo de forma formatada (como uma matriz), por exemplo:

1000F

1002F

1003F

1004F

1005F

1006F

1007F